16/03

__/__/__

⇒ Algorithm Quick-Sort(A, low, high)

Input :
A → array
low → starting index
high → ending index

Output :
Sorted array A

Step 1 : if low < high then
Step 2 : p ← Partition (A, low, high)
Step 3 : Quick-Sort(A, low, p-1)
Step 4 : Quick-Sort(A, p+1, high)
Step 5 : end if

End Algorithm

Algorithm Partition(A, low, high)

Step 1 : pivot ← A[low]
Step 2 : i ← low+1
Step 3 : j ← high

Step 4: while TRUE do

while $i \leq high$ AND $A[i] \leq pivot$ do

$i \leftarrow i+1$

while $A[j] > pivot$ do

$j \leftarrow j+1$

if $i < j$ then

swap $A[i]$ and $A[j]$

else

~~break~~

~~end while~~

~~Step 5:~~ swap $A[low]$ and $A[j]$

step 6: return j

END Algorithm

Rajiff

code for quick sort :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int partition( int A[], int lb, int ub) {
    int pivot, i, j, temp;
    i = lb +1;
    j = ub;
    while ( i<j) {
        while ( i<= ub && A[i] <= pivot) {
            i++;
        while ( A[j] > pivot)
            j--;
        if ( i<j) {
            temp = A[i];
            A[i] = A[j];
            A [j] = temp;
        }
    }
    A[lb] = A[j];
    A[j] = pivot;
    return j;
}
```

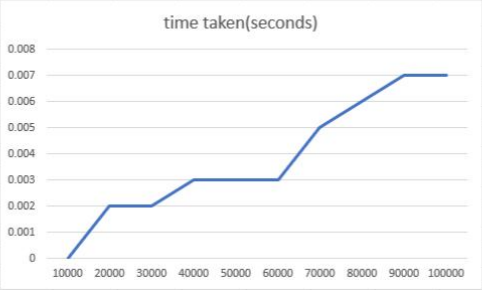
```
void quickSort(int A[], int lb, int ub){
    int k;
    if (lb < ub) {
        k = partition(A, lb, ub);
        quickSort(A, lb, k-1);
        quickSort(A, k+1, ub);
    }
}


int main(){
    int n, i;
    clock_t start, end;
    double time_taken;
    srand(time(NULL));
    for(n=10000; n<=50000; n+=10000){
        int A[n];
        for(i=0; i<=n; i++){
            A[i] = rand() % 10000;
        start = clock();
        quickSort(A, 0, n-1);
        end = clock();
        time_taken = (double)(end-start)/CLOCKS_PER_SEC;
        printf("n = %d \t Time = %d \n", n, time_taken);
    }
    return 0;
}
```

o/p :

n = 10000	Time = 0.0000
n = 20000	Time = 0.0020
n = 30000	Time = 0.0020
n = 40000	Time = 0.0020
n = 50000	Time = 0.0030

```
N       Time Taken (seconds)
10000   0.000000000000000
20000   0.002000000000000
30000   0.002000000000000
40000   0.002000000000000
50000   0.003000000000000
60000   0.003000000000000
70000   0.005000000000000
80000   0.006000000000000
90000   0.006000000000000
100000  0.007000000000000

Process returned 0 (0x0)   execution time : 0.049 s
Press any key to continue.
```

N	time taken(seconds)
10000	0
20000	0.002
30000	0.002
40000	0.003
50000	0.003
60000	0.003
70000	0.005
80000	0.006
90000	0.007
100000	0.007